МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод хорд

Студентка гр. 4342 Волкова В.Н.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Используя программы - функции HORDA и Round из файла methods.cpp, найти корень уравнения f(x) = 0 с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленности метода.

Основные теоретические положения.

Пусть найден отрезок [a, b], на котором функция f(x) меняет знак. Для определенности положим f(a)>0, f(b)<0. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения f(x) = 0 принимаются значения c_0, c_1, . . . точек пересечения хорды с осью абсцисс, как это показано на рис.1

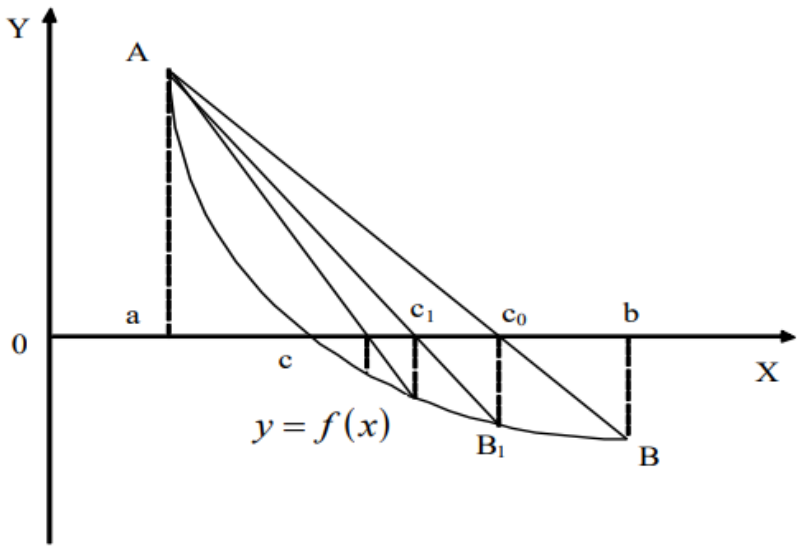

Рис. 1

Сначала находится уравнение хорды AB:

$$\frac{y-f(a)}{f(b)-f(a)}=\frac{x-a}{b-a}.$$

Для точки пересечения ее с осью абсцисс ($x=c_0$, $y=0$) получается уравнение

$$c_0=a-\frac{b-a}{f(b)-f(a)}f(a).$$

Далее сравниваются знаки величин f(a) и $f(c_0)$ и для рассматриваемого случая оказывается, что корень находится в интервале (a, c_0), так как f(a)f(c0)<0. Отрезок $[c_0,b]$ отбрасывается. Следующая итерации состоит в определении нового приближения c_1 как точки пересечения хорды AB_1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение $f(c_n)$ не станет по модулю меньше заданного числа.

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

Выполнение работы.

Вариант 2: функция $f(x) = \arccos(x^2) - x$

1. Определим отрезок [a,b]: f(x) определена при $|x|\leq 1$, то есть на отрезке $[-1,1]$. Сначала рассмотрим отрицательную часть области определения. При $x\in[-1,0]$ имеем:

$$\arccos(x^2)\geq 0,\ -x\geq 0,$$

следовательно, $f(x)=\arccos(x^2)-x>0$. Значит, на отрезке $[-1,0]$ корней нет.

Теперь рассмотрим промежуток [0,1]. Производная функции равна:

$$f'(x) = -\frac{2x}{\sqrt{1-x^4}} - 1$$

Для всех $x\in[0,1)$ выполняется

$$\frac{2x}{\sqrt{1-x^4}} \geq 0,$$

поэтому $f'(x)<0$. Следовательно, функция f(x) строго убывает на отрезке [0,1].

Проверим значения функции на концах отрезка:

$$f(0)=\arccos(0)-0=\pi/2>0,$$

$$f(1)=\arccos(1)-1=0-1=-1<0.$$

Так как функция непрерывна на [0,1] и меняет знак на концах отрезка, по теореме Больцано–Коши на этом отрезке существует корень уравнения

f(x)=0. Поскольку функция на [0,1] строго убывает, этот корень единственный.

Таким образом, в качестве отрезка локализации корня можно взять [a,b]=[0,1].

В этом можно убедиться графически, построив график f(x) (Рисунок 1).

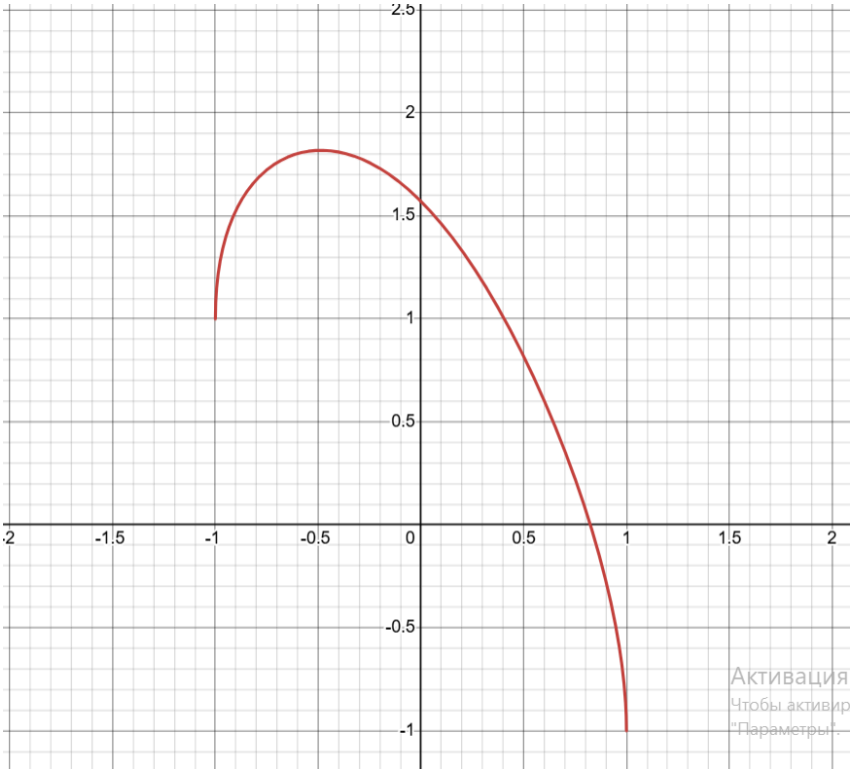

Рисунок 1 — график функции f(x)

2. Проверим условие сходимости метода хорд: Метод хорд сходится, если функция f(x) непрерывна на отрезке [a,b] и значения функции на концах имеют разные знаки: f(a)f(b)<0. В нашем случае функция $f(x)=\arccos(x^2)-x$ непрерывна на [0,1], а также f(0)>0,f(1)<0, то есть f(0)f(1)<0. Следовательно, для выбранного отрезка [0,1] метод хорд сходится.
3. В файле lb4.cpp была переопределена функция extern double F(double). Теперь она возвращает acos(x*x) - x , если точность global_delta не задана. Если задана, то перед возвратом значение округляется функцией Round. Исходный код в Приложении А.
4. Создана функция int main(), где запускаются все вычисления. Они поделены на 2 части: исследование скорости сходимости метода хорд по

зависимости N от Eps и исследование обусловленности метода по влиянию Delta на найденный корень.

5. Проведены вычисления, построен график скорости сходимости метода.

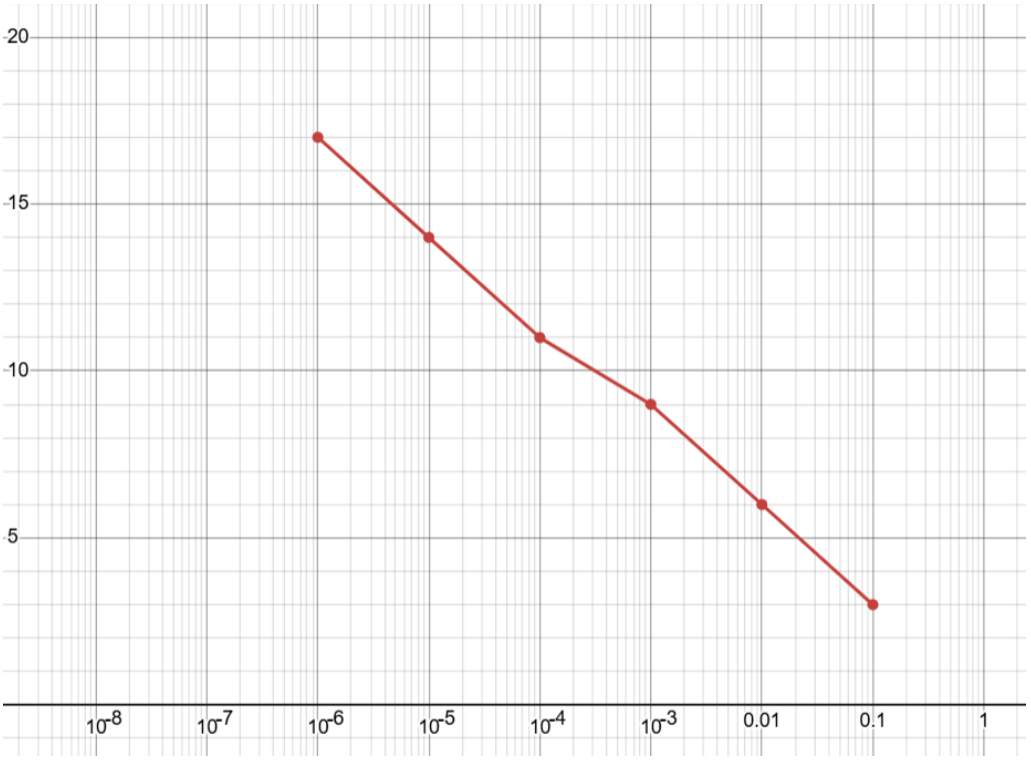

Рисунок 2 — график зависимости N от Eps

По результатам вычислений видно, что при уменьшении Eps число итераций возрастает: для Eps=0,1 требуется 3 итерации, а для Eps=10^{-6} — 17 итераций. Это подтверждает сходимость метода и показывает, что для рассматриваемой задачи скорость сходимости метода хорд является линейной: при повышении требуемой точности число итераций растет достаточно равномерно.

6. Для исследования обусловленности в функцию F(x) было добавлено округление результата с точностью Delta. Эксперимент показывает, что при грубом округлении дальнейшее уменьшение Eps уже не улучшает найденный корень, поскольку итоговая точность начинает определяться не требуемой точностью метода, а погрешностью вычисления самой функции. Ниже в Таблице 1 приведены результаты вычислений.

Таблица 1 — исследование обусловленности метода

global_delta	ε	**значение root, возвращаемое HORDA**	**N**	**F(root)**
10^{-1}	10^{-1}	0.81789	3	0.0000000000
	10^{-2}	0.81789	3	0.0000000000
	10^{-3}	0.81789	3	0.0000000000
	10^{-4}	0.81789	3	0.0000000000
	10^{-5}	0.81789	3	0.0000000000
	10^{-6}	0.81789	3	0.0000000000
10^{-3}	10^{-1}	0.796836	3	0.0860000000
	10^{-2}	0.822093	6	0.0070000000
	10^{-3}	0.824035	8	0.0000000000
	10^{-4}	0.824035	8	0.0000000000
	10^{-5}	0.824035	8	0.0000000000
	10^{-6}	0.824035	8	0.0000000000
10^{-5}	10^{-1}	0.796823	3	0.0860600000
	10^{-2}	0.822123	6	0.0065100000
	10^{-3}	0.823974	9	0.0005100000
	10^{-4}	0.82412	12	0.0000400000
	10^{-5}	0.824133	14	0.0000000000
	10^{-6}	0.824133	14	0.0000000000
10^{-7}	10^{-1}	0.796824	3	0.0860576000
	10^{-2}	0.822123	6	0.0065080000
	10^{-3}	0.823974	9	0.0005127000
	10^{-4}	0.824103	11	0.0000944000

	10^{-5}	0.82413	14	0.0000074000
	10^{-6}	0.824132	17	0.0000006000

Из таблицы видно, что при Delta=10^{-1} метод практически сразу теряет возможность уточнять решение: во всех опытах получается одно и то же приближение. При Delta=10^{-3} значение корня стабилизируется примерно на уровне 0,82403, а при Delta=10^{-5} и Delta=10^{-7} результат уже близок к точному. Следовательно, обусловленность метода напрямую связана с точностью вычисления f(x): чем меньше Delta, тем надежнее и точнее итоговый ответ.

7. Сравнение метода бисекции и метода хорд: По сравнению с методом бисекции метод хорд для данного уравнения показывает более высокую скорость сходимости. Если в методе бисекции число итераций растет по закону $N \approx \log_2((b-a)/Eps)$, то в методе хорд требуемое число итераций оказалось меньше при тех же значениях точности. Это связано с тем, что новое приближение в методе хорд определяется как точка пересечения хорды с осью абсцисс, что обычно дает более быстрое приближение к корню, чем простое деление отрезка пополам.

Выводы.

В ходе лабораторной работы с использованием функций HORDA и ROUND из файла methods.cpp был найден корень уравнения $\arccos(x^2)-x=0$ методом хорд. Для данной функции аналитически установлен рабочий интервал [0,1], на котором выполняются условия применимости метода: функция непрерывна, монотонно убывает и меняет знак на концах отрезка.

Исследование скорости сходимости показало, что при уменьшении Eps от 10^{-1} до 10^{-6} число итераций возросло с 3 до 17, что соответствует линейной сходимости метода хорд. Исследование обусловленности показало, что итоговая точность решения ограничивается точностью вычисления функции: если Delta

грубее, чем требуемая Eps, дальнейшее уменьшение Eps практически не влияет на результат.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb4.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include "methods.h"

using namespace std;

double global_delta = 0.0;

double F(double x)
{
    double y = acos(x * x) - x;
    if (global_delta > 0.0) return Round(y, global_delta);
    return y;
}

int main()
{
    double left = 0.0, right = 1.0;
    int N;

    cout << fixed << setprecision(10);

    cout << "Зависимость N от Eps" << endl;
    for (double eps = 0.1; eps >= 1e-6; eps /= 10.0)
    {
        global_delta = 0.0;
        double root = HORDA(left, right, eps, N);
        cout << "Eps: " << eps
             << "\tN: " << N
             << "\tRoot: " << root
             << "\tF(root): " << F(root) << endl;
    }

    cout << "Влияние Delta" << endl;
    global_delta = 1e-7;
    cout << "Delta: " << global_delta << endl;

    for (double eps = 0.1; eps >= 1e-6; eps /= 10.0)
    {
        double root = HORDA(left, right, eps, N);
        cout << "Eps: " << eps
             << "\tN: " << N
             << "\tRoot: " << root
             << "\tF(root): " << F(root) << endl;
    }


    return 0;
}
```

Файл methods.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);

double BISECT(double Left,double Right,double Eps,int &N)
{
  double E = fabs(Eps)*2.0;
  double FLeft = F(Left);
  double FRight = F(Right);
  double X = (Left+Right)/2.0;
  double Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  while ((Right-Left)>=E)
  {
      X = 0.5*(Right + Left);     /* вычисление середины отрезка       */
      Y = F(X);
      if (Y == 0.0) return (X);
        if (Y*FLeft < 0.0)
           Right=X;
        else
         { Left=X; FLeft=Y; }
        N++;
  };
  return(X);
}

double Round (double X,double Delta)
{
 if (Delta<=1E-9) {puts("Неверное задание точности округления\n");exit(1);}
 if (X>0.0) return (Delta*(long((X/Delta)+0.5)));
    else    return (Delta*(long((X/Delta)-0.5)));
}

double ITER(double X0,double Eps,int &N)
{
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit (1);}
  double X1=F(X0);
  double X2=F(X1);
  N = 2;
  while( (X1 - X2)*(X1 - X2) > fabs((2*X1-X0-X2)*Eps) )
  {
      X0 = X1;
      X1 = X2;
      X2 = F(X1);
        N++;
  }
   return(X2);
}

#ifdef __NEWTON
double NEWTON (double X,double Eps,int &N)
{
  extern double F1 (double);
```

```
  double Y,Y1,DX;
  N=0;
  do
  {
    Y  = F(X);
    if (Y==0.0) return (X);

    Y1 = F1(X);
    if (Y1==0.0) {puts("Производная обратилась в ноль\n");exit(1);}

    DX=Y/Y1; X=X-DX; N++;
  }
  while (fabs(DX)>Eps);
  return (X);
}
#endif

double HORDA(double Left,double Right,double Eps,int &N)
{
  double FLeft = F(Left);
  double FRight = F(Right);
  double X,Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  do
  {
      X = Left-(Right-Left)*FLeft/(FRight-FLeft);
      Y = F(X);
      if (Y == 0.0) return (X);
        if (Y*FLeft < 0.0)
         { Right=X; FRight=Y; }
        else
         { Left=X; FLeft=Y; }
        N++;
  }
  while ( fabs(Y) >= Eps );

  return(X);

}
```

Файл methods.h

```
extern double F(double);
/*****************************************************************/
/*           Функция F (X) , задаваемая пользователем            */
/*****************************************************************/

#ifdef __NEWTON
extern double F1(double);
/*****************************************************************/
/*     Производная функции F (X) , задаваемая пользователем      */
/*****************************************************************/
#endif

double Round (double X,double Delta);
```

```
/*****************************************************************/
/*   Функция Round (X,Delta) , предназначена для округления       */
/*                    X с точностью Delta                          */
/*****************************************************************/

double BISECT(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция BISECT предназначена для решения уравнения F(X)=0   */
/*    методом деления отрезка пополам. Использованы обозначения:  */
/*      Left - левый конец промежутка                              */
/*      Right - правый конец промежутка                            */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

double ITER(double X0,double Eps,int &N);
/*****************************************************************/
/*    Функция ITER предназначена для решения уравнения F(X)=X     */
/*      методом простой итерации. Использованы обозначения:       */
/*      X0 - начальное приближение корня                           */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

double HORDA(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция HORDA предназначена для решения уравнения F(x)=0    */
/*      методом хорд. Использованы обозначения:                   */
/*      Left - левый конец промежутка                              */
/*      Right - правый конец промежутка                            */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

double NEWTON (double X,double Eps,int &N);
/*****************************************************************/
/*  Функция NEWTON предназначена для решения уравнения F(X)=0     */
/*        методом касательных. Использованы обозначения:          */
/*      X - начальное приближение корня                            */
/*      Eps - погрешность вычисления корня уравнения;              */
/*      N - число итераций                                         */
/*****************************************************************/

#include "methods.cpp"
```